

CHAPTER 03

OpenCV 주요 클래스

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

3장 OpenCV 주요 클래스

3.1 기본 자료형 클래스

3.2 Mat 클래스

3.3 Vec과 Scalar 클래스

3.4 InputArray와 OutputArray 클래스

3.2 Mat 클래스

Mat 객체에 영상저장하기

- 디지털 영상은 2차원 행렬로 볼 수 있음 -> Mat 클래스 객체에 저장가능



```
170, 170, 171, 172, 172, 172, 175, 176, 178, 179, 179, 178, 177, 177, 176, 176, 176, 175, 173, 170;
170, 171, 172, 173, 174, 174, 176, 177, 178, 178, 178, 176, 175, 174, 174, 175, 174, 174, 172, 170;
171, 172, 173, 174, 175, 176, 176, 177, 177, 177, 176, 175, 173, 172, 173, 173, 173, 173, 171, 169;
171, 172, 173, 175, 176, 176, 176, 177, 177, 177, 175, 174, 172, 170, 172, 173, 173, 172, 171, 169;
171, 173, 176, 180, 180, 179, 181, 180, 177, 174, 171, 170, 169, 169, 168, 167, 166, 166, 167, 169;
171, 173, 177, 180, 180, 179, 177, 175, 173, 170, 168, 167, 167, 167, 166, 166, 165, 165, 167, 169;
171, 174, 178, 181, 180, 179, 171, 169, 167, 165, 164, 163, 164, 164, 164, 164, 164, 165, 167, 169;
```

픽셀(화소) 데이터

Mat 객체에 영상저장하기

```
Mat imread(const String& filename, int flags = IMREAD_COLOR);
```

- **filename** 불러올 영상 파일 이름
- **flags** 영상 파일 불러오기 옵션 플래그, ImreadModes 열거형 상수를 지정합니다.
- **반환값** 불러온 영상 데이터(Mat 객체)

ImreadModes 열거형 상수	설명
IMREAD_COLOR	3채널 BGR 컬러 영상으로 변환하여 불러옵니다.
IMREAD_GRAYSCALE	1채널 그레이스케일 영상으로 변환하여 불러옵니다.

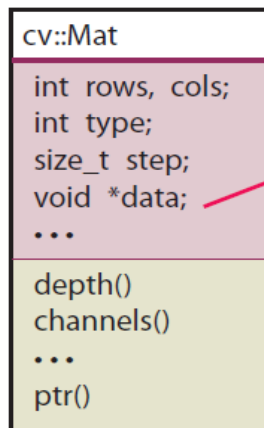
```
Mat img;
img = imread("lenna.bmp", IMREAD_COLOR);
if ( img.empty( ) ) {
    cerr << "Image load failed!" << endl;
    return -1;
}
```

Mat 객체에 영상저장하기

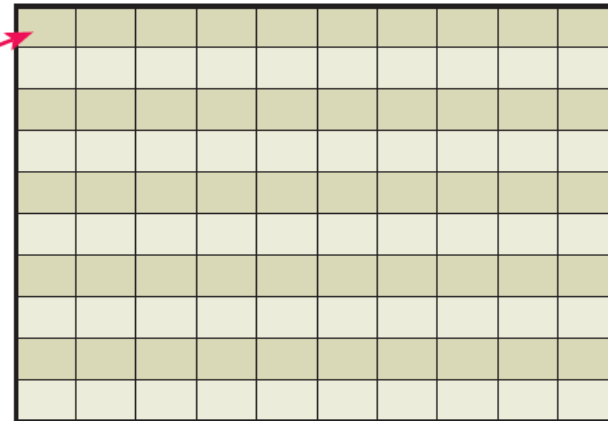
- 영상파일 dog.bmp를 불러와서 Mat 객체에 저장하는 방법

```
Mat img1 = imread("dog.bmp");
```

- Mat 객체에는 영상파일의 헤더정보만 저장되고 픽셀(화소)데이터는 동적할당을 이용하여 힙영역에 저장한다.

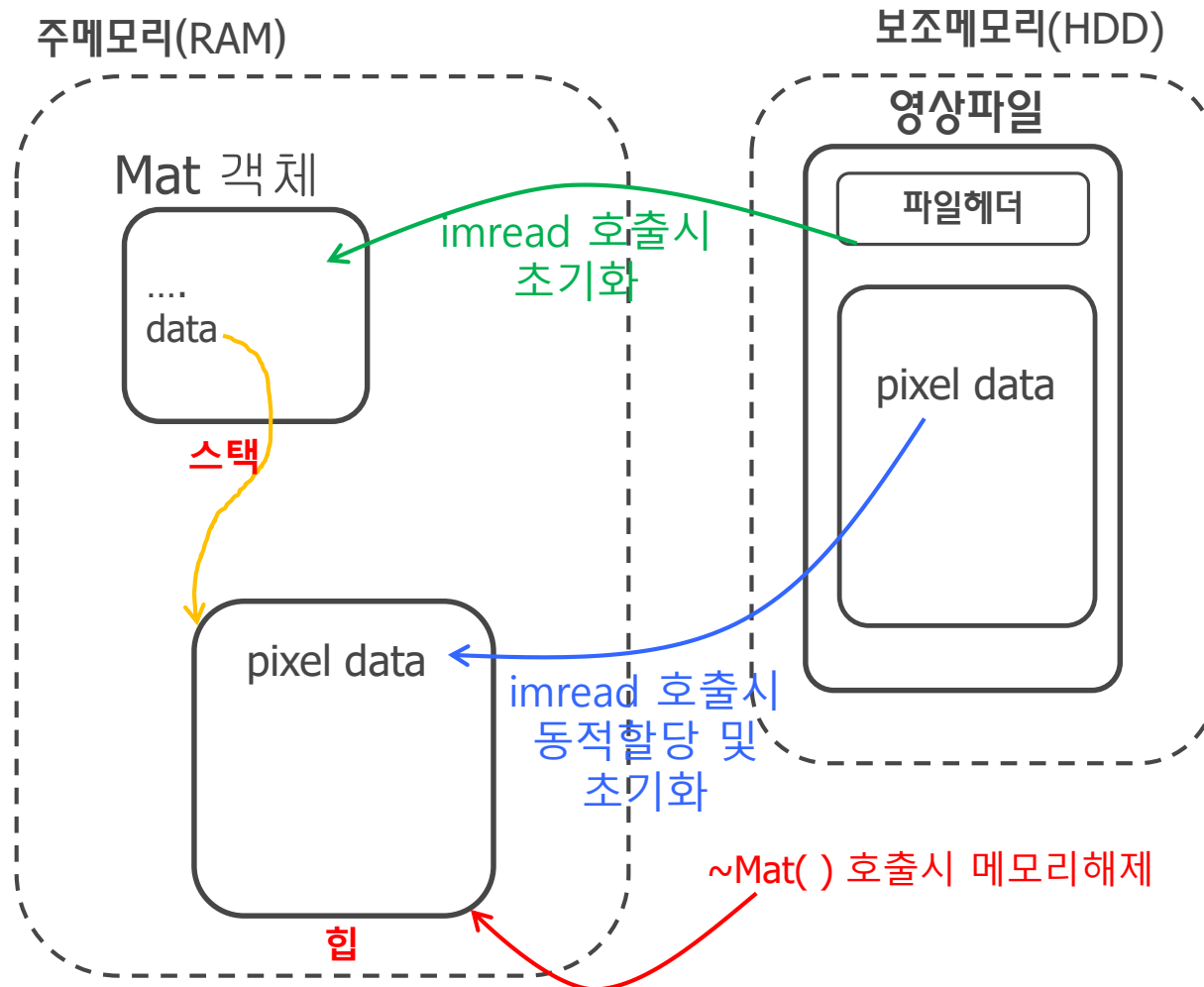


img1



픽셀(화소)데이터
-> 행렬의 원소데이터

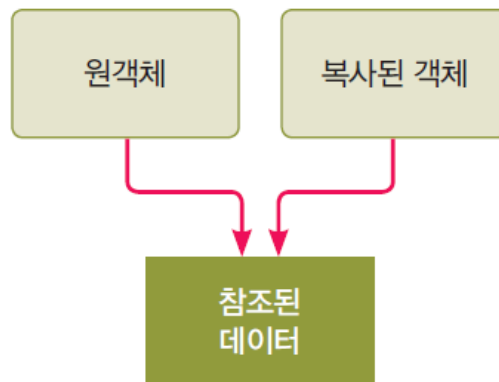
Mat 객체에 영상저장하기



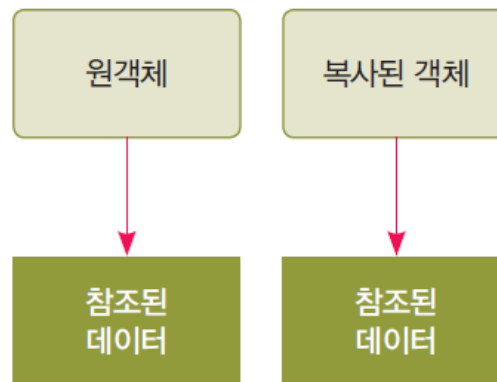
Mat 객체의 복사

- 얇은 복사(shallow copy) : Mat객체만 새로 생성되고 행렬의 원소데이터(픽셀값)를 공유하는 형태의 복사 -> 초기화 또는 대입연산자 사용시 발생.
- 깊은 복사(deep copy) : Mat객체를 복사할 때 메모리 공간을 새로 할당하여 행렬의 원소데이터(픽셀값)을 복사하는 형태의 복사 -> 전용함수를 이용해야 함

얇은 복사



깊은 복사

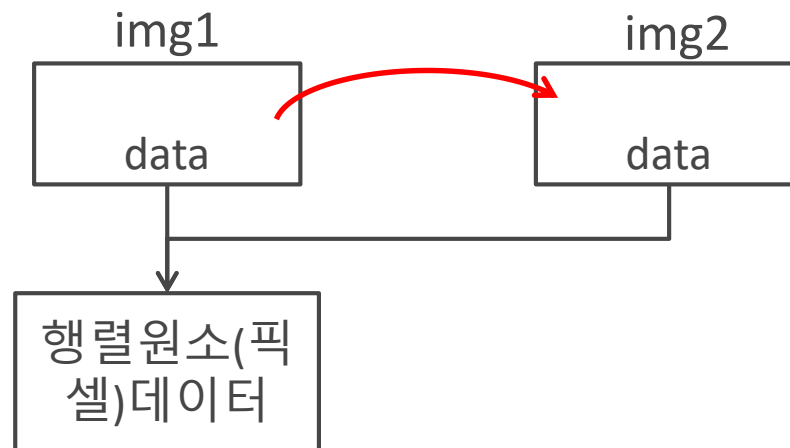


얕은 복사

- 초기화를 이용하여 img1 객체를 새로운 img2 객체에 복사

```
Mat img2 = img1;           // 복사 생성자(얕은 복사)
```

- Mat 객체로 초기화하면 행렬의 원소 데이터를 공유하는 얕은 복사(shallow copy)를 수행함
- img1과 img2는 하나의 행렬(영상)을 공유하는 서로 다른 이름의 변수 형태로 동작함

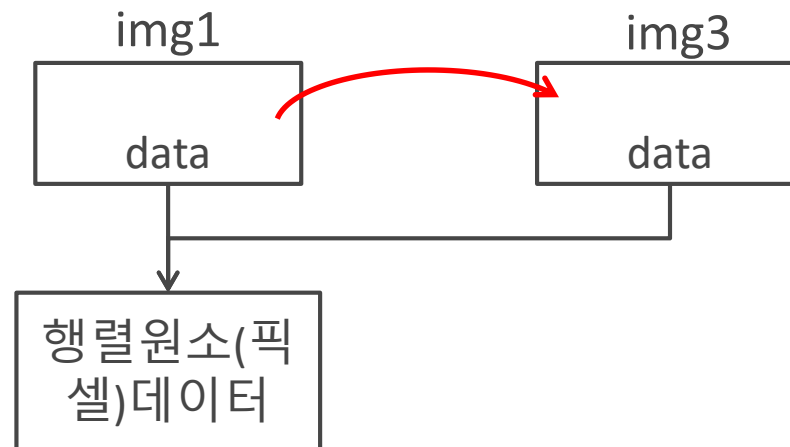


얕은 복사

- 대입 연산자를 이용하여 img1 객체를 새로운 img3 객체에 복사

```
Mat img3;  
img3 = img1;           // 대입 연산자(얕은 복사)
```

- Mat 클래스의 대입 연산자는 행렬의 원소 데이터를 공유하는 얕은 복사(shallow copy)를 수행함
- img1과 img3는 하나의 행렬(영상)을 공유하는 서로 다른 이름의 변수 형태로 동작함



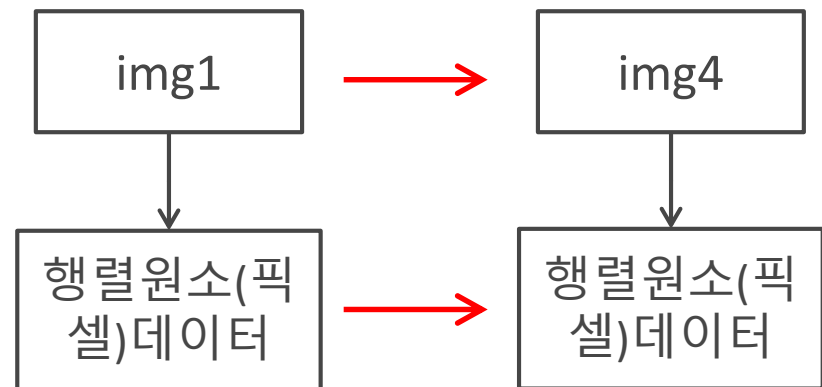
깊은 복사

```
Mat Mat::clone() const;
```

• 반환값 *this 행렬의 복사본

- Mat::clone() 함수는 자기 자신과 동일한 Mat 객체를 완전히 새로 만들어서 반환함
- Mat::clone() 함수는 메모리 공간을 새로 할당하여 픽셀 값을 복사하는 깊은 복사(deep copy)를 수행함

```
Mat img4 = img1.clone();
```



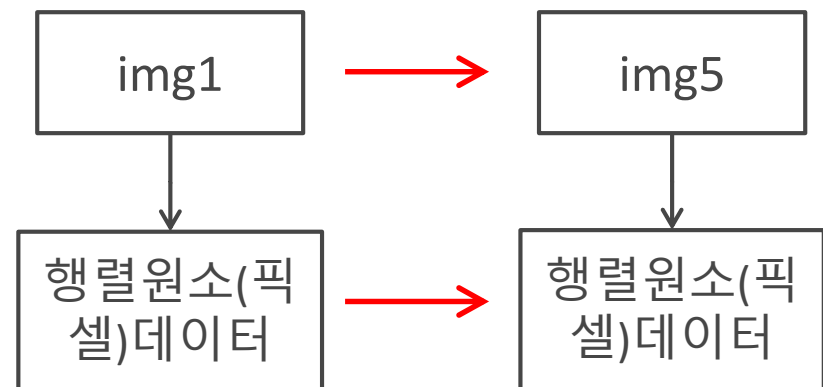
깊은 복사

```
void Mat::copyTo(OutputArray m) const;
void Mat::copyTo(OutputArray m, InputArray mask) const;
```

- **m** 복사본이 저장될 행렬. 만약 *this 행렬과 크기 및 타입이 다르면 메모리를 새로 할당한 후 픽셀 값을 복사합니다.
- **mask** 마스크 행렬. 마스크 행렬의 원소 값이 0이 아닌 좌표에서만 행렬 원소를 복사합니다.

- Mat::copyTo() 함수는 인자로 전달된 Mat 객체에 자기 자신을 복사함
- Mat::clone() 함수처럼 메모리 공간을 새로 할당하여 픽셀 값을 복사하는 깊은 복사(deep copy)를 수행함

```
Mat img5;
img1.copyTo(img5);
```



코드3-8 : 얇은/깊은 복사

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img1 = imread("dog.bmp", IMREAD_COLOR);
    if(img1.empty()) { cerr << "Image load failed!" << endl; return -1; }

    Mat img2 = img1;           // 얇은 복사
    Mat img3;
    img3 = img1;              // 얇은복사

    Mat img4 = img1.clone();   // 깊은복사
    Mat img5;
    img1.copyTo(img5);        // 깊은복사

    img1 = Scalar(0, 255, 255); //img1의 모든 원소에 값 대입
    // img1.setTo(Scalar(0, 255, 255)); //img1의 모든 원소에 인자값 대입
}
```

코드3-8 : 얇은/깊은 복사

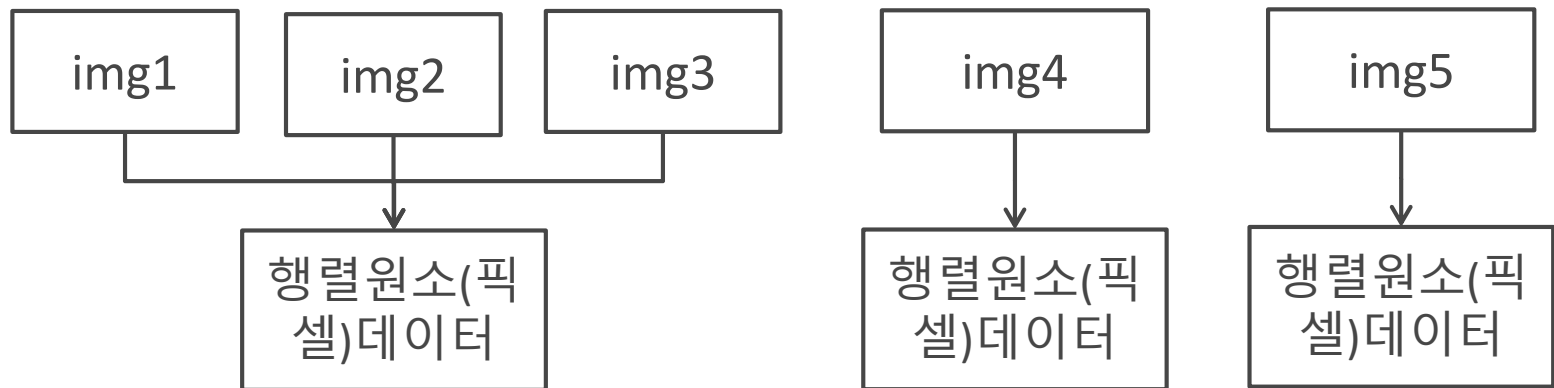
```
imshow("img1", img1);  
imshow("img2", img2);  
imshow("img3", img3);  
imshow("img4", img4);  
imshow("img5", img5);
```

```
waitKey();  
return 0;
```

```
}
```

코드3-8 : 얇은/깊은 복사

그림 3-3 행렬의 다양한 복사 방법 예제 실행 결과

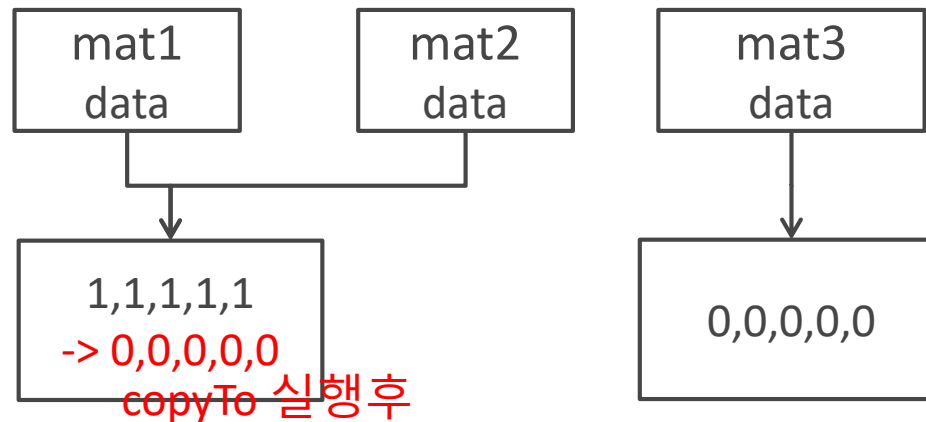


clone vs. copyTo

- copyTo함수는 원소데이터만 복사하고 Mat 객체의 data 멤버변수는 변경하지 않음, 빈 객체라면 새로운 값으로 초기화

```
Mat mat1 = Mat::ones(1, 5, CV_32F);
Mat mat2 = mat1;
Mat mat3 = Mat::zeros(1, 5, CV_32F);
mat3.copyTo(mat1);
cout << mat1 << endl;
cout << mat2 << endl;
```

```
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
```

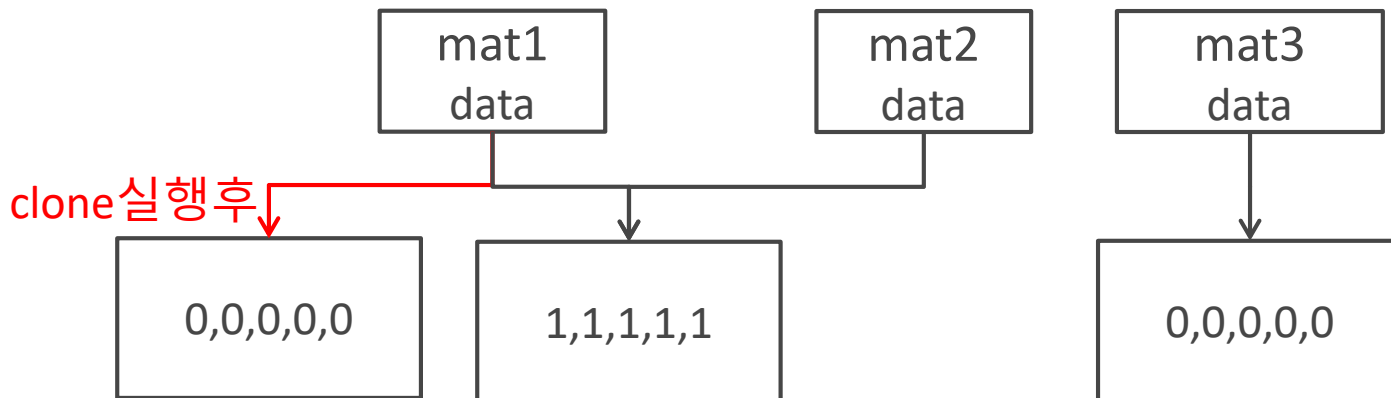


clone vs. copyTo

- clone 함수는 원소 데이터와 Mat 객체의 data 멤버 변수를 모두 변경시킨다.

```
Mat mat1 = Mat::ones(1, 5, CV_32F);
Mat mat2 = mat1;
Mat mat3 = Mat::zeros(1, 5, CV_32F);
mat1 = mat3.clone();
cout << mat1 << endl;
cout << mat2 << endl;
```

```
[0, 0, 0, 0, 0]
[1, 1, 1, 1, 1]
```



부분 행렬 추출

```
Mat Mat::operator()(const Rect& roi) const;
Mat Mat::operator()(Range rowRange, Range colRange) const;
```

- **roi** 사각형 관심 영역
- **rowRange** 관심 행 범위
- **colRange** 관심 열 범위
- **반환값** 추출한 부분 행렬 또는 영상. 부분 영상의 픽셀 데이터를 서로 공유합니다.

- Mat 클래스로 정의된 행렬에서 특정 사각형 영역의 부분 행렬을 추출하고 싶을 때에는 Mat 클래스에 정의된 괄호 연산자 함수를 사용함

171, 172, 173, 174, 174, 176, 177, 178, 178, 178, 176, 175, 174, 174, 175, 174, 174, 172, 170;	
172, 173, 174, 175, 176, 176, 177, 177, 177, 176, 175, 173, 172, 173, 173, 173, 171, 169;	
172, 173, 175, 176, 176, 176, 177, 177, 177, 175, 174, 172, 170, 172, 173, 173, 172, 171, 169;	
173, 176, 180, 180, 179, 181, 180, 177, 174, 171, 170, 169, 169, 168, 167, 166, 166, 167, 169;	177, 177, 177, 175, 174,
173, 177, 180, 180, 179, 177, 175, 173, 170, 168, 167, 167, 167, 166, 166, 165, 165, 167, 169	180, 177, 174, 171, 170,
174, 178, 181, 180, 179, 171, 169, 167, 165, 164, 163, 164, 164, 164, 164, 164, 165, 167, 169;	

img1

img2

부분 행렬 추출

- cat.bmp 파일에 저장된 고양이 영상의 (220, 120) 좌표부터 340×240 크기만큼의 사각형 부분 영상을 추출하는 코드

```
Mat img1 = imread("cat.bmp");
Mat img2 = img1(Rect(220, 120, 340, 240));
```

Mat객체(Rect객체)
피연산자 연산자

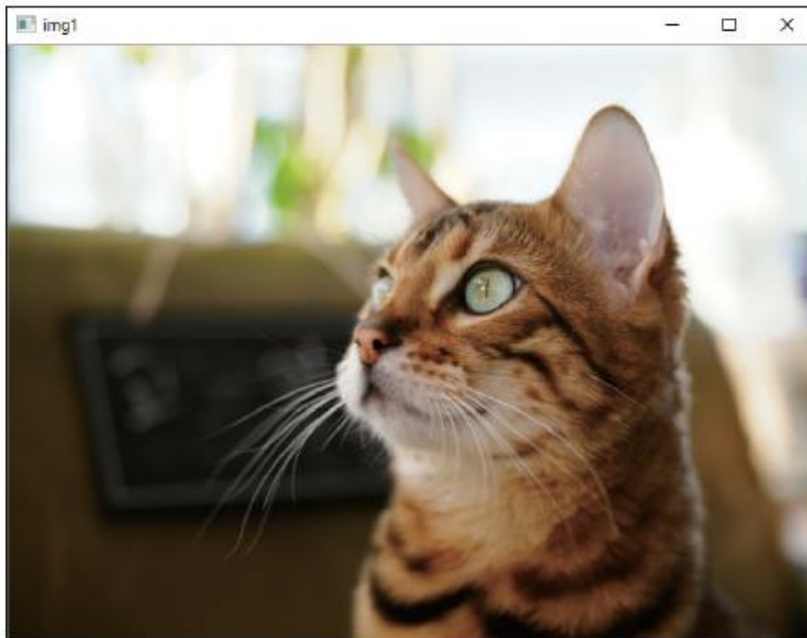
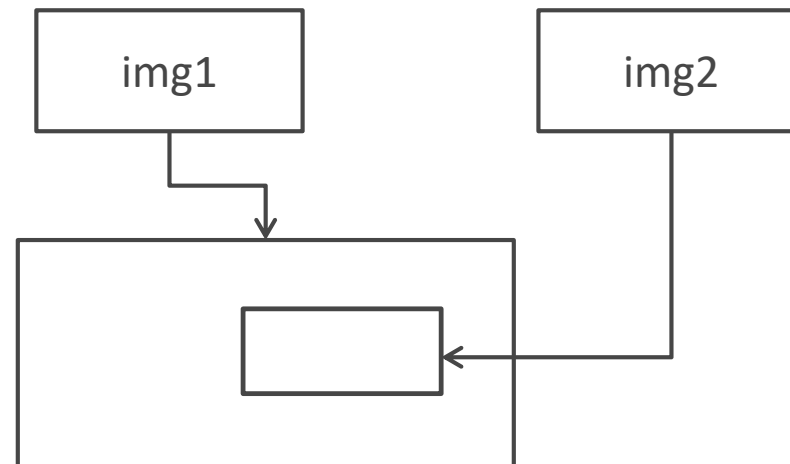


그림 3-4 부분 영상 추출 결과

부분 행렬 추출

- Mat 클래스의 괄호 연산자를 이용하여 얻은 부분 영상이 독립된 메모리 공간을 확보하여 복사하는 깊은 복사가 아니라, 픽셀 데이터를 공유하는 얇은 복사 형식이라는 점임
- 부분 영상을 추출한 후 부분 영상의 픽셀 값을 변경하면 추출한 부분 영상뿐만 아니라 원본 영상의 픽셀 값도 함께 변경됨
- 부분 영상 추출 시 픽셀 데이터를 공유한다는 특성을 이용하면 입력 영상의 일부분에만 특정한 영상 처리를 수행할 수 있음



부분 행렬 추출

- 영상의 반전(~)은 밝은 픽셀은 어둡게 만들고, 어두운 픽셀은 밝게 변화시키는 기법임 (6장 239페이지 참고, 이진수변환후 1->0, 0->1)
- img2 영상 전체가 반전이 되었고, 더불어 img1 영상에서 고양이 얼굴 주변의 부분 영상만 반전되어 나타나는 것을 확인할 수 있음

```
img2 = ~img2;
```

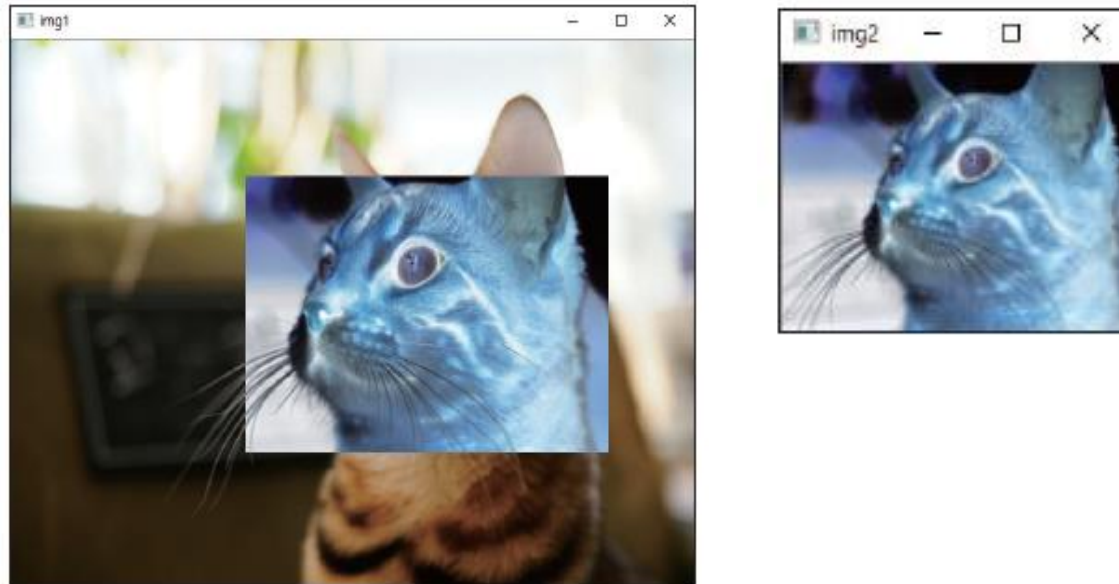


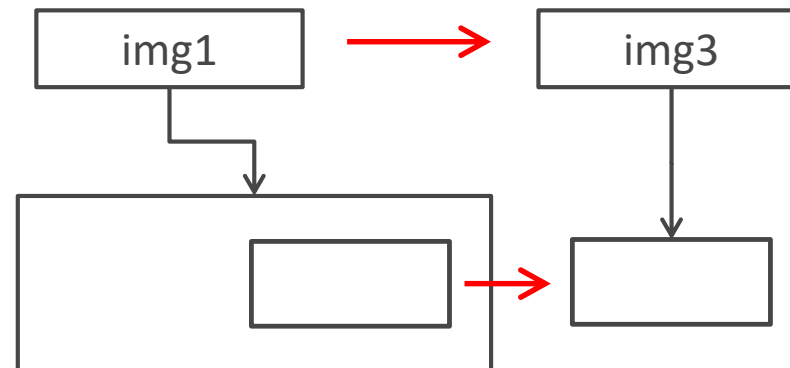
그림 3-5 부분 영상 추출 후 영상 반전 결과

부분 행렬 추출

- 관심영역(ROI, Region Of Interest)은 영상의 전체 영역 중에서 특정 영역에 대해서만 영상 처리를 수행할 때 설정하는 영역을 의미함
- Mat 클래스의 부분 행렬 추출 기능은 입력 영상에서 사각형 모양의 관심 영역(ROI) 을 설정하는 용도로 사용할 수 있음
- 만약 독립된 메모리 영역을 확보하여 부분 영상을 추출하고자 한다면 괄호 연산자 뒤에 Mat::clone() 함수를 함께 사용해야 함

```
Mat img3 = img1(Rect(220, 120, 340, 240)).clone();
```

- img1 영상과 img3 영상은 서로 다른 메모리 공간을 사용함 -> img3 영상의 픽셀 값을 변경해도 img1 영상은 변경되지 않음



부분 행렬 추출

```
Mat Mat::rowRange(int startrow, int endrow) const;  
Mat Mat::rowRange(const Range& r) const;
```

```
Mat Mat::row(int y) const;  
Mat Mat::col(int x) const;
```

```
Mat Mat::colRange(int startcol, int endcol) const;  
Mat Mat::colRange(const Range& r) const;
```

- 자세한 내용은 교재 참고

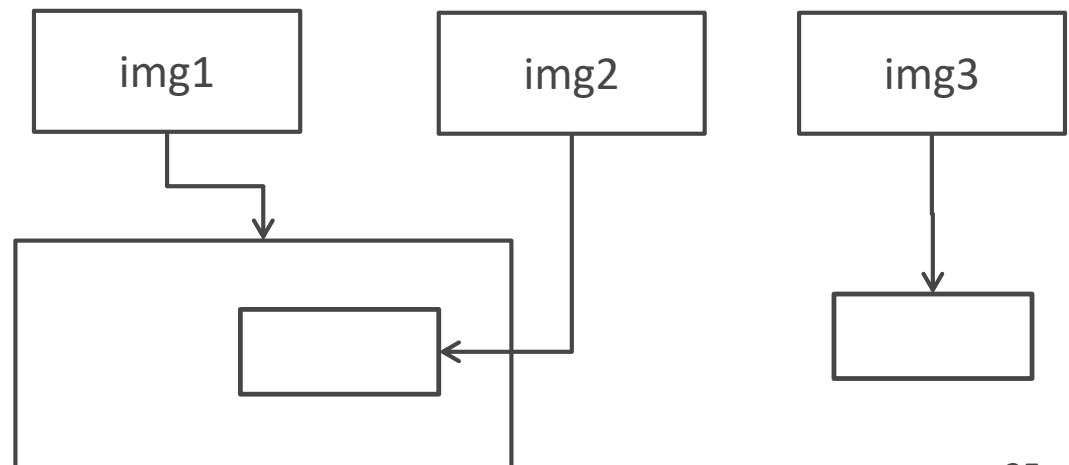
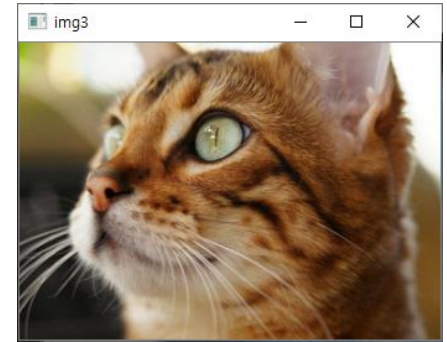
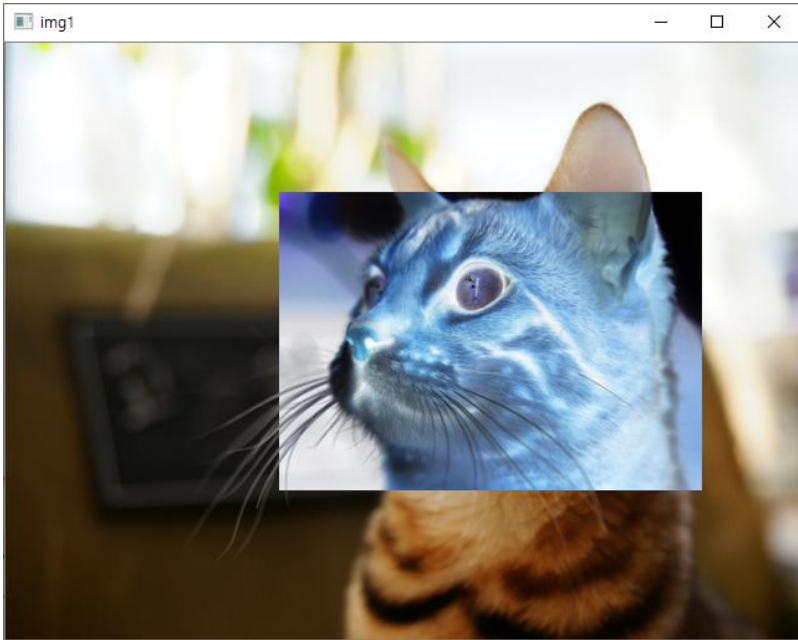
코드3-9 : 부분 영상 편집

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img1 = imread("cat.bmp");
    if (img1.empty()) { cerr << "Image load failed!" << endl; return -1;}

    Mat img2 = img1(Rect(220, 120, 340, 240));
    Mat img3 = img1(Rect(220, 120, 340, 240).clone();

    img2 = ~img2;                                //비트별not -> 이진수로 표현후 0->1, 1->0
    imshow("img1", img1);
    imshow("img2", img2);
    imshow("img3", img3);
    waitKey();
    return 0;
}
```


코드3-9 : 부분 행렬 추출



실습과제 1

- 얇은 복사의 장점, 단점을 쓰시오.
- 깊은 복사의 장점, 단점을 쓰시오.

실습과제2

- 다음 그림과 같은 원소값을 갖는 img1 객체를 생성하고 img1 행렬에서 빨간색 사각형부분의 부분행렬을 추출하여 img2에 저장하고 아래 처럼 화면에 출력 하시오.

```

Microsoft Visual Studio 디버그 콘솔

img1
[ 1, 2, 3, 4, 5;
  6, 7, 8, 9, 10;
 11, 12, 13, 14, 15]

img2
[ 8, 9, 10;
 13, 14, 15]
  
```

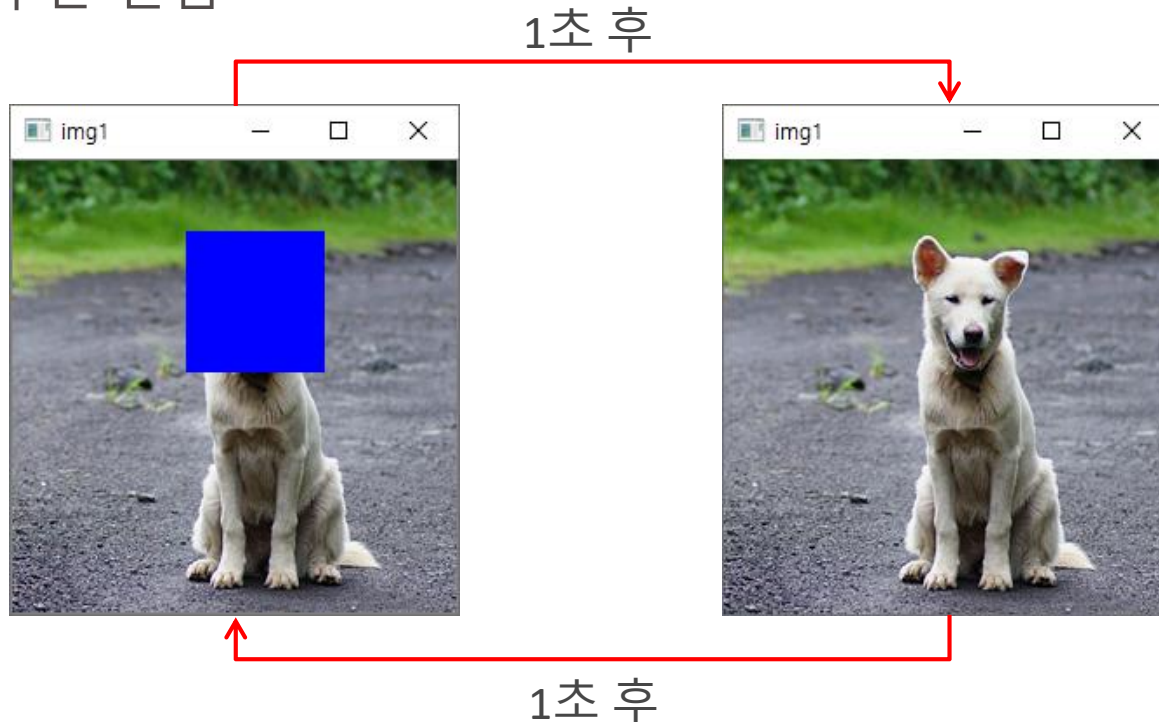
실습과제3

- 코드 3-8에서 사용한 강아지 영상파일에서 얼굴부분을 파란색으로 수정하라. 얼굴부분의 좌표값은 그림판 프로그램을 이용하여 구하시오.



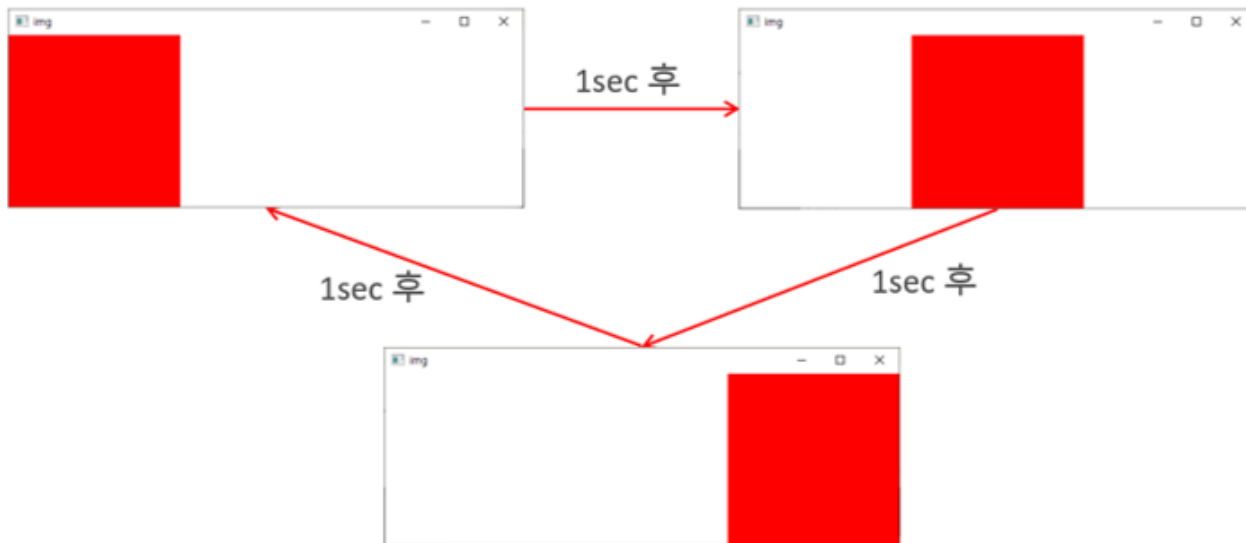
실습과제4

- 코드 3-8에서 사용한 강아지 영상파일을 다음처럼 1초간격으로 변경하는 프로그램을 작성하시오. 결과는 동영상으로 제출할 것
- 힌트: 먼저 얼굴영역을 깊은 복사로 따로 저장해 두어야 함, 얇은 복사로 얼굴영역을 Blue로 변경한 후 1초 후 깊은 복사(copyTo)로 다시 복구 시킨다. 이후 1초 주기로 같은 과정을 반복한다.
clone함수는 안됨



실습과제5(선택문제)

- 다음 조건을 만족시키는 프로그램을 작성하라.(21년 중간고사)
 - ① 아래 영상의 크기는 높이가 200픽셀, 폭이 600픽셀이다.
 - ② 아래 그림처럼 1sec마다 영상의 배경이 무한히 바뀌도록 하시오.
 - ③ 키보드에서 q가 입력되면 프로그램을 종료하도록 하시오.
 - ④ 같은 코드가 2번이상 반복되면 감점됨

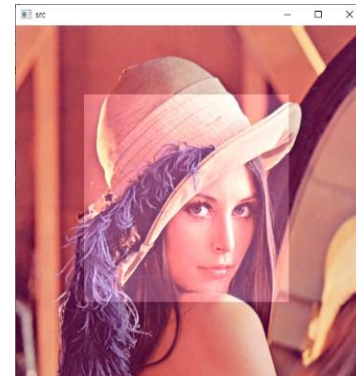


실습과제6(선택문제)

- 다음 지시대로 관심영역의 밝기를 변경하는 프로그램을 작성하시오.(21년 중간고사)
- ① lenna 영상에서 관심영역의 좌표, 크기와 픽셀값의 변화량을 입력 받는다.
- ② lenna 영상에서 관심영역 부분을 픽셀 변화량만큼 밝기를 변경한다.

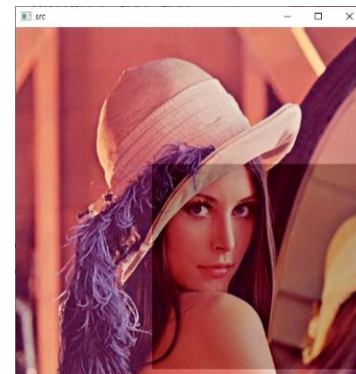
```

D:\Users\2sungryul\Dropbox\Lecture\2021년2학기\영상처리\강의노트\OpenC...
관심영역의 좌측상단좌표(x, y):100 100
관심영역의 폭,높이(width, height):300 300
픽셀변화량:50
  
```



```

D:\Users\2sungryul\Dropbox\Lecture\2021년2학기\영상처리\강의노트\OpenCV...
관심영역의 좌측상단좌표(x, y):200 200
관심영역의 폭,높이(width, height):300 300
픽셀변화량:-50
  
```



과제 제출 방법

- 소스파일, 라인단위의 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.